

Chapter 12. Assemblies and Deployment

So far in this book, I've used the term *component* to describe either a library or an executable. It's now time to look more closely at exactly what that means. In .NET the smallest unit of deployment for a software component is called an *assembly*, and it is typically a *.dll* file. Assemblies are an important aspect of the type system, because each type is identified not just by its name and namespace but also by its containing assembly. Assemblies provide a kind of encapsulation that operates at a larger scale than individual types, thanks to the `internal` accessibility specifier, which works at the assembly level.

.NET assemblies can't run on their own—they rely on the .NET runtime, and we have a few options for ensuring that a suitable runtime is available when we deploy our applications. The runtime provides an *assembly loader*, which automatically finds and loads the assemblies a program needs. To ensure that the loader can find the right components, assemblies have structured names that include version information, and they can optionally contain a globally unique element to prevent ambiguity.

Most of the C# project types in Visual Studio's "Create a new project" dialog produce a single assembly as their main output, as do most of the project templates available from the command line with `dotnet new`. When you build a project, it will often put additional files in the output folder too, such as copies of any assemblies that your code relies on that are not built into the .NET runtime, and other files needed by your application. (For example, a website project will typically need to produce CSS and script files in addition to server-side code.) But there will usually be a particular assembly that is the build target of your project, containing all of the types your project defines along with the code those types contain.

Anatomy of an Assembly

Assemblies use the Win32 Portable Executable (PE) file format, the same format that executables (EXEs) and dynamic link libraries (DLLs) have always used in modern versions of Windows.¹ It is “portable” in the sense that the same basic file format is used across different CPU architectures. Non-.NET PE files are generally architecture-specific, but .NET assemblies often aren’t. Even if you’re running .NET on Linux or macOS, it’ll still use this Windows-based format—most .NET assemblies can run on all supported operating systems, so we use the same file format everywhere.

The C# compiler produces an assembly as its output, usually with an extension of *.dll*. Tools that understand the PE file format will recognize a .NET assembly as a valid, but rather dull, PE file. The CLR essentially uses PE files as containers for a .NET-specific data format, so to classic Win32 tools, a C# DLL will not appear to export any APIs. Remember that C# compiles to a binary intermediate language (IL), which is not directly executable. The normal Windows mechanisms for loading and running the code in an executable or DLL won’t work with IL, because that can run only with the help of the CLR. Similarly, .NET defines its own format for encoding metadata and does not use the PE format’s native capability for exporting entry points or importing the services of other DLLs.

NOTE

Later, we’ll look at the ahead-of-time (AOT) compilation tools in the .NET SDK. These can incorporate native executable code into your build output, but if you enable this through the *Ready to Run* feature, even this embedded native code is loaded and executed under the control of the CLR and is directly accessible only to managed code. Native AOT is different, but we’ll get to that.

In most cases, you won’t build .NET assemblies with an extension of *.exe*. Even project types that produce directly runnable outputs (such as console or WPF applications) produce a *.dll* as their primary output. They also generate an executable file, but it’s not a .NET assembly. It’s just a *host* (sometimes referred to as a bootstrapper) that starts the runtime and then loads and executes your application’s main assembly. By default, the

type of host you get depends on what OS you build on—for example, if you build on Windows, you’ll get a Windows `.exe` host, whereas on Linux it will be an executable in the ELF format.² (The exception to this is when you target the .NET Framework. Since that supports only Windows, it doesn’t need different hosts for different operating systems, so these projects produce a .NET assembly with an extension of `.exe` that incorporates the bootstrapper.)

.NET Metadata

As well as containing the compiled IL, an assembly contains *metadata*, which provides a full description of all of the types it defines, whether public or private. The CLR needs to have complete knowledge of all the types your code uses to be able to make sense of the IL and turn it into running code—the binary format for IL frequently refers to the containing assembly’s metadata and is meaningless without it. The reflection API, which is the subject of [Chapter 13](#), makes the information in this metadata available to your code.

Resources

You can embed binary resources in a DLL alongside the code and metadata. Client-side applications might do this with bitmaps, for example. To embed a file, you can add it to a project, select it in Solution Explorer, and then use the Properties panel to set its Build Action to Embedded Resource. This embeds a copy of the entire file into the component. To extract the resource at runtime, you use the `Assembly` class’s `GetManifestResourceStream` method, which is part of the reflection API described in [Chapter 13](#). However, in practice, you wouldn’t normally use this facility directly—most applications use embedded resources through a localizable mechanism that I’ll describe later in this chapter.

So, in summary, an assembly contains a comprehensive set of metadata describing all the types it defines; it holds all of the IL for those types’ methods, and it can optionally embed any number of binary streams. This is typically all packaged up into a single PE file. However, that is not always the whole story.

Multifile Assemblies

The old (but still supported) Windows-only .NET Framework allows an assembly to span multiple files. You can split the code and metadata across multiple *modules*, and it is also possible for some binary streams that are logically embedded in an assembly to be put in separate files. This feature is rarely used, and only .NET Framework supports it. However, it's necessary to know about it because some of its consequences persist. In particular, parts of the design of the reflection API ([Chapter 13](#)) make no sense unless you know about this feature.

With a multifile assembly, there's always one main file that represents the assembly. This will be a PE file, and it contains a particular element of the metadata called the *assembly manifest*. This is not to be confused with the Win32-style manifest that most executables contain. The assembly manifest is just a description of what's in the assembly, including a list of any additional modules or other external files; in a multimodule assembly, the manifest describes which types are defined in which files.

Other PE Features

Although C# does not use the classic Win32 mechanisms for representing code or exporting APIs in EXEs and DLLs, there are still a couple of old-school features of the PE format that assemblies can use.

Win32-style resources

.NET defines its own mechanism for embedding binary resources, and a localization API built on top of that, so for the most part it makes no use of the PE file format's intrinsic support for embedding resources. Nonetheless, there's nothing stopping you from putting classic Win32-style resources into a .NET component—the C# compiler offers various command-line switches that do this. However, there's no .NET API for accessing these resources at runtime from within your application, which is why you'd normally use .NET's own resource system. But there are some exceptions.

Windows expects to find certain resources in executables. For example, it defines a way to embed version information as an unmanaged resource. C# assemblies normally do this, but you don't need to define a version resource explicitly. The compiler can generate one for you, as I show in [“Version”](#). This ensures that if an end user looks at your assembly's properties in Windows File Explorer, they will be able to see the version number. (By convention, .NET assemblies typically contain this Win32-style version information whether they target just Windows or can run on any platform.)

Windows .exe files typically contain two additional Win32 resources. You may want to define a custom icon for your application to control how it appears on the task bar or in Windows File Explorer. This requires you to embed the icon in the Win32 way, because File Explorer doesn't know how to extract .NET resources. You can do this by adding an `<ApplicationIcon>` property to your .csproj file. If you're using Visual Studio, it provides a way to set this through the project's properties pages. Also, if you're writing a classic Windows desktop application or console application (whether written with .NET or not), it should supply an application manifest. Without this, Windows will presume that your application was written before 2006³ and will modify or disable certain OS features for backward compatibility. The manifest also needs to be present if you are writing a desktop application and you want it to pass certain Microsoft certification requirements. This kind of manifest has to be embedded as a Win32 resource. The .NET SDK will add a basic manifest by default, but if you need to customize it (e.g., because you're writing a console application that will need to run with elevated privileges), you can specify a manifest with an `<ApplicationManifest>` property in your .csproj file (or again, with the project properties pages in Visual Studio).

Remember that unless you're targeting the old .NET Framework, the main assembly is a .dll, even for Windows desktop applications, and when you target Windows, the build process produces a separate .exe that launches the .NET runtime and then loads that assembly. As far as Windows is concerned, this host executable is your application, so the icon and manifest resources will end up in that file. But if you target the .NET Framework, there will be no separate host executable, so these resources end up in the main assembly.

Console versus GUI

Windows makes a distinction between console applications and Windows applications. To be precise, the PE format requires a `.exe` file to specify a *subsystem*, and back in the old days of Windows NT, this enabled the use of multiple operating system *personalities*—early versions included a POSIX subsystem, for example. These days, PE files target one of just three subsystems, and one of those is for kernel-mode device drivers. The two user-mode options used today select between Windows graphical user interface (GUI) and Windows console applications. The principal difference is that Windows will show a console window when running the latter (or if you run it from a command prompt, it will just use the existing console window), but a Windows GUI application does not get a console window.

You can select between these subsystems with an `<OutputType>` property in your project file set to `Exe` or `WinExe`, or in Visual Studio you can use the “Output type” drop-down list in the project properties. (The output type defaults to `Library`, or “Class Library” in Visual Studio’s UI. This builds a DLL, but since the subsystem is determined when a process launches, it makes no difference whether a DLL targets the Windows Console or Windows GUI subsystem. The `Library` setting always targets the former.) If you target the .NET Framework, this subsystem setting applies to the `.exe` file that is built as your application’s main assembly, and with newer versions of .NET, it will apply to the host `.exe`. (As it happens, it will also apply to the main assembly `.dll` that the host loads, but this has no effect because the subsystem is determined by the `.exe` for which the process is launched.)

Type Identity

As a C# developer, your first point of contact with assemblies will usually be the fact that they form part of a type’s identity. When you write a class, it will end up in an assembly. When you use a type from the runtime libraries or from some other library, your project will need a reference to the assembly that contains the type before you can use it.

This is not always obvious when using system types. The build system automatically adds references to various runtime library assemblies, so most of the time, you will not need to add a reference before you can use a runtime library type, and since you do not normally refer to a type's assembly explicitly in the source code, it's not immediately obvious that the assembly is a mandatory part of what it takes to pinpoint a type. But despite not being explicit in the code, the assembly has to be part of a type's identity, because there's nothing stopping you or anyone else from defining new types that have the same name as existing types. For example, you could define a class called `System.String` in your project. This is a bad idea, and the compiler will warn you that this introduces ambiguity, but it won't stop you. And even though your class will have the exact same fully qualified name as the built-in string type, the compiler and the runtime can still distinguish between these types.

Whenever you use a type, either explicitly by name (e.g., in a variable or parameter declaration) or implicitly through an expression, the C# compiler knows exactly what type you're referring to, meaning it knows which assembly defined the type. So it is able to distinguish between the `System.String` intrinsic to .NET and a `System.String` unhelpfully defined in your own component. The C# scoping rules mean that an explicit reference to `System.String` identifies the one that you defined in your own project, because local types effectively hide ones of the same name in external assemblies. If you use the `string` keyword, that always refers to the built-in type. You'll also be using the built-in type when you use a string literal, or if you call an API that returns a string. [Example 12-1](#) illustrates this—it defines its own `System.String` and then uses a generic method that displays the type and assembly name for the static type of whatever argument you pass it. (This uses the reflection API, which is described in [Chapter 13](#).)

Example 12-1. What type is a piece of string?

```
using System;

// Never do this!
namespace System
{
    public class String
```



```

    {
    }
}

class Program
{
    static void Main()
    {
        System.String? s = null;
        ShowStaticTypeNameAndAssembly(s);
        string? s2 = null;
        ShowStaticTypeNameAndAssembly(s2);
        ShowStaticTypeNameAndAssembly("String literal");
        ShowStaticTypeNameAndAssembly(Environment.OSVersion.VersionString);
    }

    static void ShowStaticTypeNameAndAssembly<T>(T item)
    {
        Type t = typeof(T);
        Console.WriteLine(
            $"Type: {t.FullName}. Assembly: {t.Assembly.FullName}.");
    }
}

```

The `Main` method in this example tries each of the ways of working with strings I just described, and it writes out the following:

```

Type: System.String. Assembly TypeIdentity, Version=1.0.0.0, Culture=neutral,
PublicKeyToken=null.
Type: System.String. Assembly System.Private.CoreLib, Version=8.0.0.0,
Culture=neutral, PublicKeyToken=7cec85d7bea7798e.
Type: System.String. Assembly System.Private.CoreLib, Version=8.0.0.0,
Culture=neutral, PublicKeyToken=7cec85d7bea7798e.
Type: System.String. Assembly System.Private.CoreLib, Version=8.0.0.0,
Culture=neutral, PublicKeyToken=7cec85d7bea7798e.

```

The explicit use of `System.String` ended up with my type, and the rest all used the system-defined string type. This demonstrates that the C# compiler can cope with multiple types with the same name. This also shows that IL is able to make that distinction. IL's binary format ensures that every reference to a type identifies the containing assembly. But just because you can create and use multiple identically named types doesn't

mean you should. You do not usually name the containing assembly explicitly in C#, so it's a particularly bad idea to introduce pointless collisions by defining, say, your own `System.String` class. (As it happens, in a pinch you can resolve this sort of collision if you really need to—see the sidebar [“Extern Aliases”](#) for details—but it's better to avoid it.)

By the way, if you run [Example 12-1](#) on .NET Framework, you'll see `mscorlib` in place of `System.Private.CoreLib`. .NET changed which assemblies many runtime library types live in. You might be wondering how this can work with .NET Standard, which enables you to write a single DLL that can run on .NET Framework and .NET. How could a .NET Standard component correctly identify a type that lives in different assemblies on different targets? The answer is that .NET has a *type forwarding* feature in which references to types in one assembly can be redirected to some other assembly at runtime. (A type forwarder is just an assembly-level attribute that describes where the real type definition can be found. Attributes are the subject of [Chapter 14](#).) .NET Standard components reference neither `mscorlib` nor `System.Private.CoreLib`—they are built as though runtime library types are defined in an assembly called `netstandard`. Each .NET runtime supplies a `netstandard` implementation that forwards to the appropriate types at runtime. In fact, even code built directly for .NET often ends up using type forwarding. If you inspect the compiled output, you'll find that it expects most runtime library types to be defined in an assembly called `System.Runtime`, and it's only through type forwarding that these end up using types in `System.Private.CoreLib`.

When multiple types with the same name are in scope, C# normally uses the one from the nearest scope, which is why a locally defined

`System.String` can hide the built-in type of the same name. It's unwise to introduce this sort of name clash, but this problem occasionally occurs when external libraries that you depend on have made bad naming decisions. If that's where you are, C# offers a mechanism that lets you specify the assembly you want. You can define an *extern alias*.

In [Chapter 1](#), I showed type aliases defined with the `using` keyword that make it easier to refer to types that have the same simple name but different namespaces. An extern alias makes it possible to distinguish between types with the same fully qualified name in different assemblies.

To define an extern alias, you need to add an `Aliases` element inside the relevant element in your `.csproj` file. Depending on whether the target component is a NuGet package, another project, or a plain DLL, that will be a `PackageReference`, `ProjectReference`, or `Reference` element, respectively. As a child of that element, add an `Aliases` element containing the name (or a comma-separated list of names) to use, e.g.,

`<Aliases>A1</Aliases>`. If you're using Visual Studio, it can do this for you: expand the Dependencies list in Solution Explorer and then expand either the Packages, Projects, or Assemblies section and select a reference. You can then set the alias for that reference in the Properties panel. If you define an alias of `A1` for one assembly and `A2` for another, you can then declare that you want to use these aliases by putting the following at the top of a C# file:

```
extern alias A1;
extern alias A2;
```

With these in place, you can qualify type names with `A1::` or `A2::` followed by the fully qualified name. This tells the compiler that you want to use types defined by the assembly (or assemblies) associated with that alias, even if some other type of the same name would otherwise have been in scope.

If it's a bad idea to have multiple types with the same name, why does .NET make it possible in the first place? In fact, supporting name collisions was not the goal; it's just a side effect of the fact that .NET makes the assembly part of the type. The assembly needs to be part of the type definition so that the CLR can know which assembly to load for you at runtime when you first use some feature of that type.

Deployment

To run the code in the assemblies that constitute our application, we need to ensure that they are copied to the computer that will host that application, along with any other supporting files necessary for successful execution. This process is called *deployment*. There are several ways to deploy a .NET application.

The first question we will need to ask when choosing a deployment strategy is: How will we ensure that a suitable .NET runtime is available on the target system? There are two possible answers: either the runtime is installed at the system level,⁴ or the application will need to bring its own copy of the runtime. So we can choose between *framework-dependent* and *self-contained* deployment models, respectively.

Framework-Dependent

The framework-dependent approach, where we rely on the .NET runtime being installed at the system level, has some advantages over a self-contained deployment. If you don't need to ship a copy of the .NET runtime, your deployable package will be much smaller because it needs to include only files specific to your application. It might also offer servicing advantages: assuming your host machines have maintenance procedures in place to install updates to the .NET runtime, you won't need to deploy new versions of your application just to receive these runtime updates. The downside, of course, is that something has to ensure that a suitable version of the .NET runtime is in fact installed.

There are some scenarios in which you can take the presence of the runtime for granted. For example, cloud platforms often preinstall it in the

virtual machines or containers that will host your code. In corporate environments, the .NET runtime might be part of standard OS images rolled out across the organization.

There are some scenarios in which you will need to take steps to ensure that the .NET runtime is installed, but for which a framework-dependent approach still makes sense. If you are using a containerization technology such as Docker, for example, Microsoft makes standard images available in which .NET is preinstalled. If you use one of these images as a starting point, your application image needs to add only the files for a framework-dependent deployment of your application.

There are two variations on the framework-dependent model. If you run this command:

```
dotnet publish
```

the SDK will build a *framework-dependent executable* deployment. This includes an executable file that makes your application runnable, sometimes referred to as the *host*. This executable's main job is to discover which .NET runtimes are installed on the target system, and to choose the most appropriate one to run your application (or to report an error if it can't find a suitable runtime). Executable files are inherently platform-specific: if you run the preceding command on Windows, you'll get a *.exe* file, but on Linux you'll get an executable that can run on Linux, and likewise with macOS. The executable's name will be the same as the name of the assembly your project produces (except the assembly has a *.dll* extension, regardless of target platform), and by default this will be the same as your project filename. So *MyApp.csproj* will build a *MyApp.dll* and an executable, which on Windows will be called *MyApp.exe*, whereas on Linux and macOS, it will be called *MyApp*.

Although publication defaults to producing an executable suitable for the OS and CPU architecture of the machine on which you run `dotnet publish`, you can ask for other targets. This command builds a framework-dependent executable where the executable file runs on macOS, and is built for the x64 processor architecture:

```
dotnet publish -r osx-x64
```

The value following the `-r` argument is a *runtime identifier* (RID). It starts with an operating system identifier (e.g., `win`, `osx`, `ios`, `linux`, or `android`; OS X has been rebranded as macOS, but the .NET SDK's Mac support predates that change). This is optionally followed by more detail, typically including the CPU architecture (e.g., `win-x86`). In Linux RIDs, it can also indicate which distribution is expected, e.g., `linux-musl-arm64`, because that will determine which system libraries are available.

This choice of RID might affect more than just the host executable. Your code could use conditional compilation (`#if` directives) to include platform-specific sections. Also, some NuGet packages support only specific RIDs, or provide RID-specific files. However, it's fairly common for everything in your deployable output to be completely portable across OS and CPU type except for the host executable.

It is possible to ask the SDK to omit the executable entirely:

```
dotnet publish -p:UseAppHost=false
```

This is the other framework-dependent style, and it is called a *framework-dependent deployment*. (That's a slightly confusing name because it's not obvious that this is distinct from a framework-dependent executable. It sounds more like the name for the broader category that includes both styles.) This produces output that does not include a host executable, but is otherwise identical to what you would get without that final argument.

How do you run your application if there's no executable? If a computer has the .NET runtime installed, the `dotnet` command-line interface (sometimes called the *dotnet CLI*) will be available on the system path. You can use `dotnet run MyApp.dll` where *MyApp.dll* is your application's main assembly, and it will run the application in exactly the same way as the host executable would have done (including working out which of the .NET runtimes installed on the system is the best choice).

The host executable adds a couple of advantages. First, it makes it easier to run the application. If you've written a command-line tool in .NET, it would be inconvenient for users of that tool to have to type `dotnet run "C:\Program Files\MyTools\MyCommand.dll"` instead of just `MyCommand`. Second, when you use system tools that monitor processes (such as the Windows Task Manager, or the `ps` command on Linux) the process will be listed as `MyCommand` if launched with a host executable. Programs launched using `dotnet run` all show up as `dotnet`, making it hard to tell which application is which if more than one program is using .NET.

There are two advantages to not using the host executable. First, as long as none of your code and none of the components you depend on require a particular OS or CPU type, a framework-dependent deployment is completely portable: the same files will work on any target platform. Second, even if you target a specific OS and CPU, omitting the application host makes your deployment slightly smaller. (The host executable for Windows on x64 processors is about 138 KB.) In some cloud platforms, minimizing the size of a deployment can reduce the startup time, so if you don't need the host executable, you may as well omit it.

Self-Contained

For scenarios where you can't presume that a suitable runtime will be installed as part of your environment, and you don't want to add any separate prerequisite procedures to install one at a system-wide level, you could choose .NET's self-contained deployment model. This bundles a complete copy of the .NET runtime (the CLR, the libraries—everything needed to run a .NET application) in the deployable output of your application. As you might expect, this makes for a much larger deployable package.

A framework-dependent deployment of a simple “Hello, World” application takes about 5.2 KB. (I'm not including the 10 KB file containing debug symbol information in that figure.) The exact size of a self-contained deployment depends on the RID, but if I build the same application with this command:

```
dotnet publish -r win-x64 --self-contained true
```

The deployable output is 70 MB in size. That’s about four orders of magnitude larger than the smallest possible framework-dependent deployment of the same code. With real applications, the difference will not be as large—as the amount of code in your application grows, and the more NuGet packages it depends on, the larger a framework-dependent deployment would grow. However, it takes a hefty application to outweigh the 70 MB of the .NET runtime.

Trimming

Once you’ve discovered just how large self-contained deployments are, you might well ask: Can I make them smaller? You can, although this creates some constraints. You can enable *trimming*, which omits code that we’re not using, whether that code resides in libraries obtained from NuGet, or the runtime libraries themselves. Enabling trimming is straightforward enough: you just add a single property to your project file, as [Example 12-2](#) shows.

Example 12-2. Enabling trimming in project file

```
<PropertyGroup>
  <PublishTrimmed>true</PublishTrimmed>
</PropertyGroup>
```

NOTE

The `PublishTrimmed` setting affects only self-contained deployment (which includes Native AOT if you’re using that). It has no impact on any other build outputs.

Enabling trimming on a “Hello, World” application targeting `win-x64` gets the size down from 70 MB to about 17.5 MB. That’s a substantial improvement, although it’s still much larger than a framework-dependent deployment. A trimmed self-contained deployment still includes a copy of

the CLR because we need basic services such as JIT compilation and garbage collection. Since the runtime makes use of its own libraries internally, a significant subset of the runtime libraries will be included even when you don't use them directly.

Trimming can be particularly important when we use the Mono CLR. Remember, .NET offers more than one CLR. The default one most applications use offers sophisticated mechanisms to deliver high performance, but these aren't always a good fit in memory-constrained environments. The Mono CLR was, for many years, optimized for mobile platforms, and now also supports in-browser execution by targeting Web Assembly (WASM). This CLR is significantly smaller, meaning that assemblies (application-specific, runtime library components, and NuGet packages) typically account for the majority of the size of a deployment. If you are building for WASM in order to run .NET code inside a web browser (e.g., when using the Blazor framework) trimming can improve page load speeds considerably.

It is not always straightforward for the build system to work out which library code is in use. For most of .NET's existence, it was safe to assume that the whole of the runtime library was available, because either .NET is installed or it is not, and so some libraries have come to depend on dynamic mechanisms such as runtime code generation. These make it tricky for the build process to work out what is safe to omit—your application code might have no direct references to a particular library type, but that type might end up being loaded via reflection, or used by code emitted at runtime.

WARNING

Trimming is often incompatible with mechanisms that rely on discovering type information at runtime using reflection (see [Chapter 13](#)). For example, certain ways of using the serialization features described in [Chapter 15](#) do this, although they also offer trimming-friendly modes of operation.

The runtime libraries include annotations to help the compiler understand how to trim the code, and so that it can warn you when you are using non-trim-compatible features. Before .NET 7.0, components that had

no such annotations were treated as untrimmable by default, but now the SDK will analyze components that are not annotated to detect the use of non-trim-friendly features. If it can determine that a component does not use such features (or that your application doesn't use the parts of that component that do), it can safely trim code.

Ahead-of-Time (AOT) Compilation

As you know, the C# compiler emits code in a non-CPU-specific form called IL, and this is normally compiled into machine code at runtime at the first moment that it is required, a process called *just-in-time* (JIT) compilation. However, you can instruct the .NET SDK to generate machine language (or, where applicable, WASM) at build time. This is known as ahead-of-time (AOT) compilation. .NET offers two forms of AOT: *ReadyToRun* and *Native AOT*.

ReadyToRun

ReadyToRun (R2R) is a hybrid model in which native code gets generated at build time and is added to the assembly alongside everything else that would normally be included. All of the IL and type information remain present. R2R is a conservative option: it will never prevent anything from working. Even if your compiled code ends up running on a different CPU architecture than the native code was generated for, it will still work because the IL is still present, so the CLR can just fall back to JIT compilation.

The main benefit of R2R is that it can improve application startup time. As long as the native code embedded in the assembly matches the target architecture, the CLR can use that instead of JIT compiling code. You can enable R2R by adding a property to your project file, as [Example 12-3](#) shows.

Example 12-3. Enabling R2R in a project file

```
<PropertyGroup>
  <PublishReadyToRun>true</PublishReadyToRun>
</PropertyGroup>
```

The R2R code generation occurs only when you publish—it’s relatively slow, so you won’t generally want to wait for it to happen during normal development and debugging. You will need to specify a RID so that the build tools know which OS and CPU architecture to build native code for:

```
dotnet publish -r win-x64
```

Even when the R2R code is a match for the target system, the CLR might choose to replace it after a while. The runtime keeps track of which methods are used most often, and for heavily used methods, it may decide to regenerate the code to better match usage patterns (a mechanism called *tiered JIT*). For example, it might notice that although a method has an argument with an interface type, in practice the same concrete type seems to be passed every time, and so it can generate a version of the method optimized for that type (while still being able to fall back to more general code in case some other type is ever passed in).

R2R does not radically change the way that code runs. It just enables the CLR to avoid having to JIT compile methods in some circumstances. Consequently, R2R code needs the .NET runtime to be present. You can use it with either self-contained or framework-dependent deployment.

Native AOT

The second form of AOT compilation is Native AOT. This is a self-contained model, and it works much more like traditional compilation of the kind used by languages such as C and C++: all machine code is generated at build time. The resulting executable contains everything it needs to run, so you can just copy the file onto a target machine and run it. (This is, in effect, a kind of self-contained deployment.) The IL is not copied into the build output, and type information will be included only where the build tools detect that it is required. JIT compilation is not available. It is enabled with a project file setting, as [Example 12-4](#) shows.

Example 12-4. Enabling Native AOT in a project file

```
<PropertyGroup>  
  <PublishAot>true</PublishAot>
```

As with R2R, Native AOT code generation occurs only when you publish the application. And again we need to specify a RID so the tools know what kind of native code is required. For example:

```
dotnet publish -r win-x64
```

Native AOT is a more radical option than R2R. Code built this way can't run on an OS or processor architecture other than the one it was built for because the IL is not included, meaning that there's no way for the CLR to fall back to JIT compilation. The version of the CLR that gets embedded in a Native AOT application does not include the JIT compiler. (This also means that tiered JIT compilation is not available, so the performance can sometimes be lower.) Native AOT trims code, so the constraints that apply to trimming also apply here—certain dynamic techniques such as run-time code generation are unavailable.

Native AOT can produce the smallest self-contained programs. (This is an area in which .NET 8.0 has made significant improvements.) Earlier, we saw that a self-contained, trimmed “Hello, world” application was 17.5 MB in size when built for `win-x64`. With Native AOT this comes down to about 1.4 MB. Those of us old enough to remember floppy disks might raise an eyebrow at the idea that the simplest possible command-line program might barely fit on a removable storage disk, but remember that this does include a CLR. (There's a garbage collector in there, for example.) Compared to conventional self-contained deployment's 70 MB (17.5 MB trimmed), 1.4 MB looks positively frugal. Native AOT enables you to produce completely self-contained executables whose size is in the same ballpark as with languages such as Go and Rust.

An application will typically start up more quickly when you use Native AOT than it will with the other deployment options. That's partly down to the small file size—smaller files can be loaded into memory more quickly. The Native AOT CLR is much smaller than the normal .NET CLR, so it loads quickly too. You also don't need to wait for JIT compilation, but there's even an advantage over R2R deployment. With R2R, the CLR still has to decide whether the available native code is suitable, and because it

needs to be able to substitute JIT compiled code where appropriate, it must add a level of indirection that Native AOT can avoid. So Native AOT applications typically have faster startup times even than R2R applications. This can make Native AOT a good choice in cloud environments that provision execution environments dynamically—systems such as AWS Lambda or Azure Functions might wait until a network request comes into a service before actually loading your code into memory and may shut it down after only a very short period of inactivity, and in these cases, startup time can have a particularly significant effect on average performance.

Loading Assemblies

You may have been alarmed earlier when I said that the build system automatically adds references to all the runtime library components available on your target framework. Perhaps you wondered how you might go about removing some of these in the name of efficiency. As far as runtime overhead is concerned, you do not need to worry. The C# compiler effectively ignores any references to built-in assemblies that your project never uses, so there's no danger of loading DLLs that you don't need. (If you're not using trimming, however, it's worth avoiding references to unused components that are *not* built into .NET to avoid copying unneeded DLLs when you deploy the app—there's no sense in making deployments larger than they need to be. So you shouldn't add references to NuGet packages that you're not using. But unused references to DLLs that are already installed as part of .NET cost you nothing.)

Even if C# didn't strip out unused references at compile time, there would still be no risk of unnecessary loading of unused DLLs. The CLR does not attempt to load assemblies until your application first needs them. Most applications do not exercise every possible code path each time they execute, so it's fairly common for significant portions of the code in your application not to run. Your program might finish its work having left entire classes unused—perhaps classes that get involved only when an unusual error condition arises. If the only place you use a particular assembly is inside a method of such a class, that assembly won't get loaded.

NOTE

Native AOT works differently because it produces a single executable file containing not just your application's code, but also all of the code it uses from libraries. We don't deploy assemblies when using Native AOT, so the assembly loading mechanisms described in this section don't apply. Instead, it just relies on the usual operating system mechanisms for demand-loading of code.

The CLR has some discretion for deciding exactly what it means to “use” a particular assembly. If a method contains any code that refers to a particular type (e.g., it declares a variable of that type or it contains expressions that use the type implicitly), then the CLR may consider that type to be used when that method first runs even if you don't get to the part that really uses it. Consider [Example 12-5](#).

Example 12-5. Type loading and conditional execution

```
static IComparer<string> GetComparer(bool useStandardOrdering)
{
    if (useStandardOrdering)
    {
        return StringComparer.CurrentCulture;
    }
    else
    {
        return new MyCustomComparer();
    }
}
```

Depending on its argument, this function returns either an object provided by the runtime libraries' `StringComparer` or a new object of type `MyCustomComparer`. The `StringComparer` type is defined in the same assembly as core types such as `int` and `string`, so that will have been loaded when our program started. But suppose the other type, `MyCustomComparer`, was defined in a separate assembly from my application, called `ComparerLib`. Obviously, if this `GetComparer` method is called with an argument of `false`, the CLR will need to load `ComparerLib` if it hasn't already. But what's slightly more surprising is that it will probably load `ComparerLib` the first time this method is called

even if the argument is `true`. To be able to JIT compile this `GetComparer` method, the CLR will need access to the `MyCustomComparer` type definition—for one thing it will need to check that the type really has a zero-argument constructor. (Obviously [Example 12-5](#) wouldn't compile in that case, but it's possible that code was compiled against a different version of `ComparerLib` than is present at runtime.) The JIT compiler's operation is an implementation detail, so it's not fully documented and could change from one version to the next, but it seems to operate one method at a time. So simply invoking this method is likely to be enough to trigger the loading of the `ComparerLib` assembly.

This raises the question of how .NET finds assemblies. If assemblies can be loaded implicitly as a result of running a method, we don't necessarily have a chance to tell the runtime where to find them. So .NET has a mechanism for this.

Assembly Resolution

When the runtime needs to load an assembly, it goes through a process called *assembly resolution*. In some cases you will tell .NET to load a particular assembly (e.g., when you first run an application), but the majority are loaded implicitly. The exact mechanism depends on whether your application is self-contained.

In a self-contained deployment, assembly resolution is pretty straightforward because everything—your application's own assemblies, any external libraries you depend on, all of the system assemblies built into .NET, and the CLR itself—ends up in one folder. So unless the application directs the CLR to look elsewhere, everything will load from the application folder, including all the .NET runtime library assemblies.

Framework-dependent applications necessarily use a more complex resolution mechanism than self-contained ones. When such an application starts up, it will first determine exactly which version of .NET to run. This won't necessarily be the version your application was built against, and there are various options to configure exactly which is chosen. By default, if the same `Major.Minor` version is available, that will be used. E.g., if a framework-dependent application built for .NET 7.0 runs on a system

with .NET versions 6.0.24, 7.0.12, and 8.0.0 installed, it will run on 7.0.12. It is also possible to run on a higher major version number than the app was built against (e.g., build for 7.0 but run on 8.0) but only by explicitly requesting this through configuration. (The build tools automatically produce a file called *YourApp.runtimeconfig.json* in your build output declaring which version you are using, and this file can include settings to enable *roll-forward*.)

The chosen runtime version selects not just the CLR but also the assemblies making up the .NET runtime libraries. You can typically find all the installed runtime versions in the *C:\Program Files\dotnet\shared\Microsoft.NETCore.App* folder on Windows, */usr/local/share/dotnet/shared/Microsoft.NETCore.App* on macOS, or */usr/share/dotnet/shared/Microsoft.NETCore.App* on Linux, with version-based subfolders such as 8.0.0. (You should not rely on these paths—the files may move in future versions of .NET.) The assembly resolution process will look in this version-specific folder, and this is how framework-dependent applications get to use built-in .NET assemblies.

If you poke around these folders, you may notice other folders under *shared*, such as *Microsoft.AspNetCore.App*. It turns out that the shared component mechanism is not just for the runtime libraries built into .NET—it is also possible to install the assemblies for whole frameworks. .NET applications declare that they are using a particular application framework. (The *YourApp.runtimeconfig.json* file in your build output declares not just the .NET version, but also the framework you are using. Console apps specify *Microsoft.NETCore.App*, whereas a web application will specify *Microsoft.AspNetCore.App*, and WPF or Windows Forms apps specify *Microsoft.WindowsDesktop.App*. The build tools automatically work out what to put in there based on what's in your project file, so you normally don't need to configure this yourself.) This enables applications that target specific Microsoft frameworks not to have to include a complete copy of all of the framework's DLLs even though that framework is not part of .NET itself.

If you install the plain .NET runtime, you will get just *Microsoft.NETCore.App* and none of the application frameworks. So applications that target frameworks such as ASP.NET Core or WPF will be

unable to run if they are built for framework-dependent deployment (which is the default) because that presumes that those frameworks will be preinstalled on target systems, and the assembly resolution process will fail to find framework-specific components. The .NET SDK installs these additional framework components, so you won't see this problem on your development machine, but you might see it when deploying at runtime. You can tell the build tools to include the framework's components, but this is not normally necessary. If you run your application on a public cloud service such as Azure, these generally preinstall relevant framework components, so in practice you will usually only run into this situation if you are configuring a server yourself or when deploying desktop applications. For those cases, Microsoft offers installers for the [.NET runtime](#) that also include the components for web or desktop frameworks.

The *shared* folder in the *dotnet* installation folder is not one you should modify yourself. It is intended only for Microsoft's own frameworks. However, it is possible to install additional system-wide components if you want, because .NET also supports something called the *runtime package store*. This is an additional directory structured in much the same way as the *shared* folder just described. You can build a suitable directory layout with the `dotnet store` command, and if you set the `DOTNET_SHARED_STORE` environment variable, the CLR will look in there during assembly resolution. This enables you to play the same trick as is possible with Microsoft's frameworks: you can build applications that depend on a set of components without needing to include them in your build output, as long as you've arranged for those components to be preinstalled on the target system.

Aside from looking in these two locations for common frameworks, the CLR will also look in the application's own directory during assembly resolution, just as it would for a self-contained application. Also, the CLR has some mechanisms for enabling updates to be applied. For example, on Windows, it is possible for Microsoft to push out critical updates to .NET components via Windows Update.

But broadly speaking, the basic process of assembly resolution for framework-dependent applications is that implicit assembly loading occurs ei-

ther from your application directory or from a shared set of components installed on the system. This is also true for applications running on the older .NET Framework, although the mechanisms are a bit different. It has something called the *Global Assembly Cache* (GAC), which effectively combines the functionality provided by both of the shared stores in .NET. It is less flexible, because the store location is fixed; .NET's use of an environment variable opens up the possibility of different shared stores for different applications.

Explicit Loading

Although the CLR will load assemblies automatically, you can also load them explicitly. For example, if you are creating an application that supports plug-ins, during development you will not know exactly what components you will load at runtime. The whole point of a plug-in system is that it's extensible, so you'd probably want to load all the DLLs in a particular folder. (You would need to use reflection to discover and make use of the types in those DLLs, as [Chapter 13](#) describes.)

WARNING

Native AOT has no JIT compiler, so you can use only those assemblies that were compiled to native code during the build. Some of the APIs discussed in this section will work, but only for assemblies your project depends on in the conventional way, and only if trimming hasn't discarded the relevant code. This doesn't support scenarios that need truly dynamic loading, such as plug-in systems.

If you know the full path of an assembly, loading it is very straightforward: you call the `Assembly` class's static `LoadFrom` method, passing the path of the file. (This method is completely unsupported in Native AOT.) The path can be relative to the current directory, or it can be absolute. This static method returns an instance of the `Assembly` class, which is part of the reflection API. It provides ways of discovering and using the types defined by the assembly.

Occasionally, you might want to load a component explicitly (e.g., to use it via reflection) without wanting to specify the path. For example, you

might want to load a particular assembly from the runtime libraries. You should never hardcode the location for a system component—they tend to move from one version of .NET to the next. If your project has a reference to the relevant assembly and you know the name of a type it defines, you can write `typeof(TheType).Assembly`. But if that's not an option, you should use the `Assembly.Load` method, passing the name of the assembly.

`Assembly.Load` uses exactly the same mechanism as implicitly triggered loading. So you can refer to either a component that you've installed alongside your application or a system component. In either case, you should specify a full name (see [“Assembly Names”](#)) e.g., `ComparerLib, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null`.

WARNING

If you've enabled trimming, this approach might not work if you use a particular type only through dynamic loading and reflection, because the trimmer might not understand that you are using a particular type.

The .NET Framework version of the CLR remembers which assemblies were loaded with `LoadFrom`. If an assembly loaded in this way triggers the implicit loading of further assemblies, the CLR will search the location from which that assembly was loaded. This means that if your application keeps plug-ins in a separate folder that the CLR would not normally look in, those plug-ins could install other components that they depend on in that same plug-in folder. The CLR will then find them without needing further calls to `LoadFrom`, even though it would not normally have looked in that folder for an implicitly triggered load. However, .NET does not support this behavior. It provides a different mechanism to support plug-in scenarios.

Isolation and Plug-ins with AssemblyLoadContext

.NET provides a type called `AssemblyLoadContext`. It enables a degree of isolation between groups of assemblies within a single application.⁵ This

solves a problem that can arise in applications that support a plug-in model.

If a plug-in depends on some component that the hosting application also uses, but wants a different version than the host, this can cause problems if you use the simple mechanisms described in the preceding section.

Typically, the .NET runtime *unifies* these references, meaning that it loads just a single version. In any cases where the types in that shared component are part of the plug-in interface, this is exactly what you need: if an application requires plug-ins to implement some interface that relies on types from, say, the `Newtonsoft.Json` library, it's important that the application and the plug-ins all agree on which version of that library is in use.

But unification can cause problems with components used as implementation details, and not as part of the API between the application and its plug-ins. If the host application uses, say, v6.0 of `Microsoft.Extensions.Logging` internally, and a plug-in uses v8.0 of the same component, there's no particular need to unify this to a single version choice at runtime—there would be no harm in the application and plug-in each using the version they require. Unification could cause problems: forcing the plug-in to use v6.0 would cause exceptions at runtime if it attempted to use features only present in v8.0. Forcing the application to use v8.0 could also cause problems because major version number changes often imply that a breaking change was introduced.

To avoid these kinds of problems, you can introduce custom assembly load contexts. You can write a class that derives from `AssemblyLoadContext`, and for each of these that you instantiate, the .NET runtime creates a corresponding load context that supports loading of different versions of assemblies than may already have been loaded by the application. You can define the exact policy you require by overloading the `Load` method, as [Example 12-6](#) shows.

Example 12-6. A custom `AssemblyLoadContext` for plug-ins

```
using System.Reflection;
using System.Runtime.Loader;
```

```

namespace HostApp;

public class PlugInLoadContext(
    string pluginPath,
    ICollection<string> plugInApiAssemblyNames) : AssemblyLoadContext
{
    private readonly AssemblyDependencyResolver _resolver = new(pluginPath);
    private readonly ICollection<string> _plugInApiAssemblyNames =
        plugInApiAssemblyNames;

    protected override Assembly Load(AssemblyName assemblyName)
    {
        if (!_plugInApiAssemblyNames.Contains(assemblyName.Name!))
        {
            string? assemblyPath = _resolver.ResolveAssemblyToPath(assemblyName)
            if (assemblyPath != null)
            {
                return LoadFromAssemblyPath(assemblyPath);
            }
        }

        return AssemblyLoadContext.Default.LoadFromAssemblyName(
            assemblyName);
    }
}

```

This takes the location of the plug-in DLL, along with a list of the names of any special assemblies where the plug-in must use the same version as the host application. (This would include assemblies defining types used in your plug-in interface. You don't need to include assemblies that are part of .NET itself—these are always unified, even if you use custom load contexts.) The runtime will call this class's `Load` method each time an assembly is loaded in this context. This code checks to see whether the assembly being loaded is one of the special ones that must be common to plug-ins and the host application. If not, this looks in the plug-in's folder to see if the plug-in has supplied its own version of that assembly. In cases where it will not use an assembly from the plug-in folder (either because the plug-in hasn't supplied this particular assembly or because it is one of the special ones), this context defers to `AssemblyLoadContext.Default`, meaning that the application host and plug-in use the same assemblies in these cases. [Example 12-7](#) shows this in use.

Example 12-7. Using the plug-in load context

```
Assembly[] plugInApiAssemblies =  
[  
    typeof(IPlugIn).Assembly,  
    typeof(JsonReader).Assembly  
];  
var plugInAssemblyNames = new HashSet<string>(  
    plugInApiAssemblies.Select(a => a.GetName().Name!));  
  
var ctx = new PlugInLoadContext(plugInDllPath, plugInAssemblyNames);  
Assembly plugInAssembly = ctx.LoadFromAssemblyPath(plugInDllPath);
```

This builds a list of assemblies that the plug-in and application must share, and passes their names into the plug-in context, along with a path to the plug-in DLL. Any DLLs that the plug-in depends on and that are copied into the same folder as the plug-in will be loaded, unless they are in that list, in which case the plug-in will use the same assembly as the host application itself.

Assembly Names

Assembly names are structured. They always include a *simple name*, which is the name by which you would normally refer to the DLL, such as *MyLibrary* or *System.Runtime*. This is usually the same as the filename but without the extension. It doesn't technically have to be,⁶ but the assembly resolution mechanism assumes that it is. Assembly names always include a version number. There are also some optional components, including the *public key token*, a string of hexadecimal digits, which makes it possible to give an assembly a unique name.

Strong Names

If an assembly's name includes a public key token, it is said to be a *strong name*. Microsoft advises that any .NET component that targets .NET Framework and is published for shared use (e.g., made available via NuGet) should have a strong name. However, if you are writing a new component that will only run on .NET, there are no benefits to strong

naming, because these newer runtimes never validate the public key token.

Since the purpose of strong naming is to make the name unique, you may be wondering why assemblies do not simply use a Globally Unique Identifier (GUID). The answer is that historically, strong names also did another job: they were designed to provide some degree of assurance that the assembly has not been tampered with. Early versions of .NET checked strongly named assemblies for tampering at runtime, but these checks were removed because they imposed a considerable runtime overhead, often for little or no benefit. Microsoft's documentation now explicitly advises against treating strong names as a security feature. However, in order to understand and use strong names, you need to know how they were originally meant to work.

As the terminology suggests, an assembly name's public key token has a connection with cryptography. It is the hexadecimal representation of a 64-bit hash of a public key. Strongly named assemblies are required to contain a copy of the full public key from which the hash was generated. The assembly file format also provides space for a digital signature, generated with the corresponding private key.

If you're not familiar with asymmetric encryption, this is not the place for a thorough introduction, but here's a very rough summary. Strong names use an encryption algorithm called RSA, which works with a pair of keys: the public key and the private key. Messages encrypted with the public key can be decrypted only with the private key, and vice versa. This enables the creation of a digital signature for an assembly: to sign an assembly, you calculate a hash of its contents and then encrypt that hash with the private key. This signature is then copied into the assembly, and its validity can be verified by anyone with access to the public key—they can calculate the hash of the assembly's contents themselves, and they can decrypt your signature with the public key, and if the results are different, the signature is invalid, implying either that it was not produced by the owner of the private key or that the file has been modified since the signature was generated, so the file is suspect. The mathematics of encryption are such that it is thought to be essentially impossible to create a valid-looking signature unless you have access to the private key, and it's also essentially impossible to modify the assembly without modifying the hash. And in cryptography, “essentially impossible” means “theoretically possible but too computationally expensive to be practical, unless some major unexpected breakthrough in number theory or perhaps quantum computing emerges, rendering most current cryptosystems useless.”

The uniqueness of a strong name relies on the fact that key generation systems use cryptographically secure random-number generators, and the chances of two people generating two key pairs with the same public key token are vanishingly small. The assurance that the assembly has not been tampered with comes from the fact that a strongly named assembly must be signed, and only someone in possession of the private key can generate a valid signature. Any attempt to modify the assembly after signing it will invalidate the signature.

The signature associated with a strong name is independent of Authenticode, a longer-established code signing mechanism in Windows. These serve different purposes. Authenticode provides traceability, because the public key is wrapped in a certificate that tells you something

about where the code came from. With a strong name's public key token, all you get is a number, so unless you happen to know who owns that token, it tells you nothing. Authenticode lets you ask, "Where did this component come from?" A public key token lets you say, "This is the component I want." It's common for a single .NET component to use both mechanisms.

If an assembly's private key becomes public knowledge, anyone can generate valid-looking assemblies with the corresponding key token. Some open source projects deliberately publish both keys so that anyone can build the components from source. This completely abandons any security the key token could offer, but that's fine because Microsoft now recommends that we not treat strong names as a security feature. The practice of publishing your strong naming private key recognizes that it is useful to have a unique name, even without a guarantee of authenticity. .NET took this one step further, by making it possible for components to have a strong name without needing to use a private key at all. In keeping with Microsoft's adoption of open source development, this means you can now build and use your own versions of Microsoft-authored components that have the same strong name, even though Microsoft has not published its private key. See the sidebar, ["Strong Name Keys and Public Signing,"](#) for information on how to work with keys.

There are three popular approaches for working with strong names. The simplest is to use the real names throughout the development process and to copy the public and private keys to all developers' machines so that they can sign the assemblies every time they build. This approach is viable only if you don't want to keep the private key secret, because it's easy for developers to compromise the secrecy of the private key either accidentally or deliberately. Since strong names no longer offer security, there's nothing wrong with this. Some organizations nonetheless attempt to keep their private keys secret as a matter of policy, so you may encounter other ways of working.

Another approach is to use a completely different set of keys during development, switching to the real name only for designated release builds. This avoids the need for all developers to have a copy of the real private key, but it can cause confusion, because developers may end up with two sets of components on their machines, one with development names and one with real names.

The third approach is to use the real names across the board, but instead of signing every build, just filling the part of the file reserved for the signature with 0 values. .NET calls this *Public Signing*, and it's more of a convention than a feature: it works because these runtimes never check the signatures of strongly named assemblies. (.NET Framework does still check signatures in certain cases. For example, to install an assembly in the GAC, it must have a strong name with a valid signature. It has a slightly more complex mechanism called *Delay Signing*, which makes you jump through a few more hoops, but the effect is the same: developers can compile assemblies that have the real strong names without then needing to generate signatures.)

You can generate a key file for a strong name with a command-line utility called *sn* (short for *strong name*).

Microsoft uses the same token on most of the assemblies in the runtime libraries. (Many groups at Microsoft produce .NET components, so this token is common only to the components that are part of .NET, not for

Microsoft as a whole.) Here's the full name of `mscorlib`, a system assembly that offers definitions of various core types such as `System.String`:

```
mcorlib, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089
```

By the way, that's the right name even for the latest versions of .NET at the time of writing. The `Version` is `4.0.0.0` even though .NET Framework is now on v4.8.1, and .NET on 8.0. (In .NET, `mcorlib` contains nothing but type forwarders, because the relevant types have moved, mostly to `System.Private.CoreLib`. And while that real home of these types is now on version `8.0.0.0`, the `mcorlib` version number remains the same.) Assembly version numbers have technical significance, so Microsoft does not always update the version number in the names of library components in step with the marketing version numbers.

While the public key token is an optional part of an assembly's name, the version is mandatory.

Version

All assembly names include a four-part version number. When an assembly name is represented as a string (e.g., when you pass one as an argument to `Assembly.Load`), the version consists of four decimal integers separated by dots (e.g., `4.0.0.0`). The binary format that IL uses for assembly names and references limits the range of these numbers—each part must fit in a 16-bit unsigned integer (a `ushort`), and the highest allowable value in a version part is actually one less than the maximum value that would fit, making the highest legal version number `65534.65534.65534.65534`.

Each of the four parts has a name. From left to right, they are the *major version*, the *minor version*, the *build*, and the *revision*. However, there's no particular significance to any of these names. Some developers use certain conventions, but nothing checks or enforces them. A common convention is that any change in the public API requires a change to either the major or minor version number, and a change likely to break existing

code should involve a change of the major number. (Marketing is another popular reason for a major version change.) If an update is not intended to make any visible changes to behavior (except, perhaps, fixing a bug), changing the build number is sufficient. The revision number could be used to distinguish between two components that you believe were built against the same source but not at the same time. Alternatively, some people relate the version numbers to branches in source control, so a change in just the revision number might indicate a patch applied to a version that has long since stopped getting major updates. However, you're free to make up your own meanings. As far as the CLR is concerned, there's really only one interesting thing you can do with a version number, which is to compare it with some other version number—either they match or one is higher than the other.

NOTE

NuGet packages also have version numbers, and these do not need to be connected in any way to assembly versions. Many package authors make them similar by convention, but this is not universal. NuGet *does* treat the components of a package version number as having particular significance: it has adopted the widely used *semantic versioning* rules. This uses versions with three parts, named major, minor, and patch.

Version numbers in runtime library assembly names ignore all the conventions I have just described. Most of the components had the same version number (`2.0.0.0`) across four major updates. With .NET 4.0, everything changed to `4.0.0.0`, which is still in use with the latest version of .NET Framework (4.8). In .NET 8.0, many of these components now have a matching major version of 8, but as you've seen with its copy of `mscorlib`, that's not universal.

You typically specify the version number by adding a `<Version>` element inside a `<PropertyGroup>` of your `.csproj` file. (Visual Studio also offers a UI for this: if you open the Properties page for the project, its Package section lets you configure various naming-related settings. The “Package version” field sets the version.) The build system uses this in two ways: it sets the version number on the assembly, but, if you generate a NuGet package for your project, by default it will also use this same version number

for the package, and since NuGet version numbers have three parts, you normally specify just three numbers here, and the fourth part of the assembly version will default to zero. (If you really want to specify all four digits, consult the documentation for how to set the assembly and NuGet versions separately.)

The build system tells the compiler which version number to use for the assembly name via an assembly-level attribute. I'll describe attributes in more detail in [Chapter 14](#), but this one's pretty straightforward. If you want to find it, the build system typically generates a file called *ProjectName.AssemblyInfo.cs* in a subfolder of your project's *obj* folder. This contains various attributes describing details about the assembly, including an `AssemblyVersion` attribute, such as the one shown in [Example 12-8](#).

Example 12-8. Specifying an assembly's version

```
[assembly: System.Reflection.AssemblyVersion("1.0.0.0")]
```

The C# compiler provides special handling for this attribute—it does not apply it blindly as it would most attributes. It parses the version number and embeds it in the way required by .NET's metadata format. It also checks that the string conforms to the expected format and that the numbers are in the allowed range.

By the way, the version that forms part of an assembly's name is distinct from the one stored using the standard Win32 mechanism for embedding versions. Most .NET files contain both kinds. By default, the build system will use the `<Version>` setting for both, but it's common for the file version to change more frequently. This was particularly important with .NET Framework, in which only a single instance of any major version can be installed at once—if a system has .NET Framework 4.7.2 installed and you install .NET Framework 4.8.1, that will replace version 4.7.2. (.NET doesn't do this—you can install any number of versions side by side on a single computer.) This in-place updating combined with Microsoft's tendency to keep assembly versions the same across releases could make it hard to work out exactly what is installed, at which point the file version becomes important. On a computer with .NET Framework 4.0 sp1 in-

stalled, its version of *mscorlib.dll* had a Win32 version number of 4.0.30319.239 . If you've installed .NET 4.8.1, this changes to 4.8.9181.0 , but the assembly version remains at 4.0.0.0 . (As service packs and other updates are released, the file version will keep climbing.)

By default, the build system will use the `<Version>` for both the assembly and Windows file versions, but if you want to set the file version separately, you can add a `<FileVersion>` to your project file. (Visual Studio's project properties Package section also lets you set this.) Under the covers, this works with another attribute that gets special handling from the compiler, `AssemblyFileVersion` . It causes the compiler to embed a Win32 version resource in the file, so this is the version number users see if they right-click your assembly in Windows Explorer and show the file properties.

This file version is usually a more appropriate place to put a version number that identifies the build provenance than the version that goes into the assembly name. The latter is really a declaration of the supported API version, and any updates that are designed to be fully backward compatible should probably leave it unaltered and should change only the file version.

Version Numbers and Assembly Loading

Since version numbers are part of an assembly's name (and therefore its identity), they are also, ultimately, part of a type's identity. The `System.String` in `mscorlib` version 2.0.0.0 is not the same thing as the type of the same name in `mscorlib` version 4.0.0.0 .

The handling of assembly version numbers changed with .NET. In .NET Framework, when you load a strongly named assembly by name (either implicitly by using types it defines or explicitly with `Assembly.Load`), the CLR requires the version number to be an exact match.⁷ .NET relaxed this, so if the version on disk has a version number equal to or higher than the version requested, it will use it. There are two factors behind this change. The first is that the .NET development ecosystem has come to rely on NuGet (which didn't even exist for most of the first decade of .NET's existence), meaning that it has become increasingly common to de-

pend on fairly large numbers of external components. Second, the rate of change has increased—in the early days we would often need to wait for years between new releases of .NET components. (Security patches and other bug fixes might turn up more often, but new functionality would tend to emerge slowly, and typically in big chunks, as part of a whole wave of updates to the runtime, frameworks, and development tools.) But today, it can be rare for an application to go for as long as a month without the version of some component somewhere changing. .NET Framework’s strict versioning policy now looks unhelpful. (In fact, there are parts of the build system dedicated to digging through your NuGet dependencies, working out the specific versions of each component you’re using, and picking the best version to use when different components want different versions of some shared component. When you target .NET Framework, by default it will automatically generate a configuration file with a vast number of version substitution rules telling the CLR to use those versions no matter which version any single assembly says it wants. So even if you target the .NET Framework, the build system will, by default, effectively disable strict versioning.)

Another change is that .NET Framework only takes assembly versions into account for strongly named assemblies. .NET checks that the version number of the assembly on disk is equal to or greater than the required version regardless of whether the target assembly is strongly named.

Culture

So far we’ve seen that assembly names include a simple name, a version number, and optionally a public key token. They also have a *culture* component. (A culture represents a language and a set of conventions, such as currency, spelling variations, and date formats.) This is not optional, although the most common value for this is the default: `neutral`, indicating that the assembly contains no culture-specific code or data. The culture is usually set to something else only on assemblies that contain culture-specific resources. The culture of an assembly’s name is designed to support localization of resources such as images and strings. To show how, I’ll need to explain the localization mechanism that uses it.

All assemblies can contain embedded binary streams. (You can put text in these streams, of course. You just have to pick a suitable encoding.) The `Assembly` class in the reflection API provides a way to work directly with these, but it's more common to use the `ResourceManager` class in the `System.Resources` namespace. This is far more convenient than working with the raw binary streams, because the `ResourceManager` defines a container format that allows a single stream to hold any number of strings, images, sound files, and other binary items, and Visual Studio has a built-in editor for working with this container format. The reason I'm mentioning all of this in the middle of a section that's ostensibly about assembly names is that `ResourceManager` also provides localization support, and the assembly name's culture is part of that mechanism. To demonstrate how this works, I'll walk you through a quick example.

The easiest way to use the `ResourceManager` is to add a resource file in the `.resx` format to your project. (This is not the format used at runtime. It's an XML format that gets compiled into the binary format required by `ResourceManager`. It's easier to work with text than binary in most source control systems. It also makes it possible to work with these files if you're using an editor without built-in support for the format.) To add one of these from Visual Studio's Add New Item dialog, select the Visual C#→General category, and then choose Resources File. I'll call mine `MyResources.resx`. Visual Studio will show its resource editor, which opens in string editing mode, as [Figure 12-1](#) shows. As you can see, I've defined a single string with a name of `ColString` and a value of `Color`.

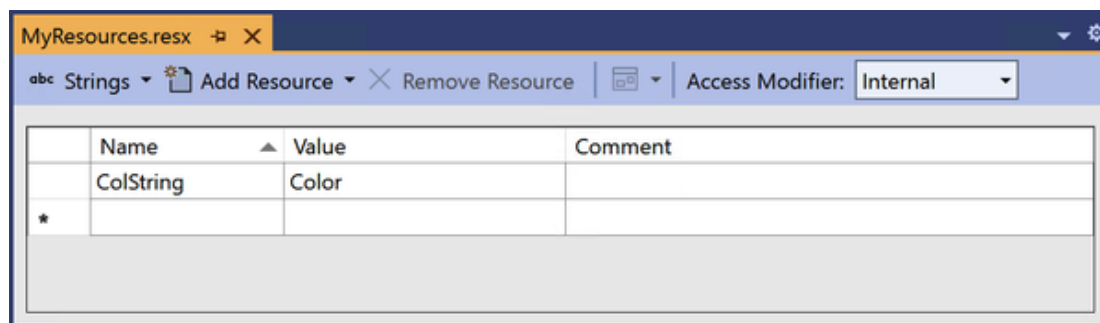


Figure 12-1. Resource file editor in string mode

I can retrieve this value at runtime. The build system generates a wrapper class for each `.resx` file you add, with a static property for each resource you define. This makes it very easy to look up a string resource, as [Example 12-9](#) shows.

Example 12-9. Retrieving a resource with the wrapper class

```
string colText = MyResources.ColString;
```

The wrapper class hides the details, which is usually convenient, but in this case, the details are the whole reason I'm demonstrating a resource file, so I've shown how to use the `ResourceManager` directly in [Example 12-10](#). I've included the entire source for the file, because namespaces are significant here—the build tools prepend your project's default namespace to the embedded resource stream name, so I've had to ask for `ResourceExample.MyResources` instead of just `MyResources`. (If I had put the resources in a subfolder, the tools would also include the name of that folder in the resource stream name.)

Example 12-10. Retrieving a resource at runtime

```
using System.Resources;

namespace ResourceExample;

class Program
{
    static void Main()
    {
        var rm = new ResourceManager(
            "ResourceExample.MyResources", typeof(Program).Assembly);
        string colText = rm.GetString("ColString")!;
        Console.WriteLine($"And now in {colText}");
    }
}
```

So far, this is just a rather long-winded way of getting hold of the string `"Color"`. However, now that we've got a `ResourceManager` involved, I can define some localized resources. Being British, I have strong opinions on the correct way to spell the word *color*. They are not consistent with O'Reilly's editorial policy, and in any case I'm happy to adapt my work for my predominantly American readership. But a program can do better—it should be able to provide different spellings for different audiences. (And taking it a step further, it should be able to change the language entirely

for countries in which some form of English is not the predominant language.) In fact, my program already contains all the code it needs to support localized spellings of the word *color*. I just need to provide it with the alternative text.

I can do this by adding a second resource file with a carefully chosen name: *MyResources.en-GB.resx*. That's almost the same as the original but with an extra *.en-GB* before the *.resx* extension. That is short for English-Great Britain, and it is the standardized (albeit politically tone-deaf) name of the culture for my home. (The name for the culture that denotes English-speaking parts of the US is *en-US*.) Having added such a file to my project, I can add a string entry with the same name as before, `ColString`, but this time with the correct (where I'm sitting⁸) value of `Colour`. If you run the application on a system configured with a British locale, it will use the British spelling. The odds are that your machine is not configured for this locale, so if you want to try this, you can add the code in [Example 12-11](#) at the very start of the `Main` method in [Example 12-10](#) to force .NET to use the British culture when looking up resources.

Example 12-11. Forcing a nondefault culture

```
Thread.CurrentThread.CurrentUICulture =  
    new System.Globalization.CultureInfo("en-GB");
```

How does this relate to assemblies? Well, if you look at the compiled output, you'll see that, as well as the usual executable file and related debug files, the build process has created a subdirectory called *en-GB*, which contains an assembly file called *ResourceExample.resources.dll*. (*ResourceExample* is the name of my project. If you created a project called *SomethingElse*, you'd see *SomethingElse.resources.dll*.) That assembly's name will look like this:

```
ResourceExample.resources, Version=1.0.0.0, Culture=en-GB, PublicKeyToken=null
```

The version number and public key token will match those for the main project—in my example, I've left the default version number, and I've not

given my assembly a strong name. But notice the `Culture`. Instead of the usual `neutral` value, I've got `en-GB`, the same culture string I specified in the filename for the second resource file I added. If you add more resource files with other culture names, you'll get a folder containing a culture-specific assembly for each culture you specify. These are called *satellite resource assemblies*.

When you first ask a `ResourceManager` for a resource, it will look for a satellite resource assembly with the same culture as the thread's current UI culture. So it would attempt to load an assembly using the name shown a couple of paragraphs ago. If it doesn't find that, it tries a more generic culture name—if it fails to find `en-GB` resources, it will look for a culture called just `en`, denoting the English language without specifying any particular region. Only if it finds neither (or if it finds matching assemblies, but they do not contain the resource being looked up) does it fall back to the neutral resource built into the main assembly.

The CLR's assembly loader looks in different places when a nonneutral culture is specified. It looks in a subdirectory named for the culture. That's why the build process placed my satellite resource assembly in an *en-GB* folder.

TIP

Since this localization mechanism depends on loading a culture-specific assembly at runtime you might expect this not to work on Native AOT. In fact, .NET makes special provisions to support this localization mechanism on Native AOT, although this is disabled by default because it adds runtime overheads: the `InvariantGlobalization` build variable defaults to `true` for Native AOT projects, so you'll need to set this to `false` in your project file to use `ResourceManager` for localization.

The search for culture-specific resources incurs some runtime costs. These are not large, but if you're writing an application that will never be localized, you might want to avoid paying the price for a feature you're not using. As just mentioned it's disabled by default for Native AOT, but you can use that same setting to disable it in any .NET application. More subtly, if the majority of your users are in one particular locale, you can

embed the resources for one particular culture directly into your main assembly, avoiding the need to load a satellite assembly but still supporting localization in other regions. You do this by putting those resources not in a culture-specific file such as *MyResources.en-US.resx*, but in a file with no culture name, such as *MyResources.resx*. You can then indicate that these are in fact the right resources for a particular locale by applying the assembly-level attribute shown in [Example 12-12](#).

Example 12-12. Specifying the culture for built-in resources

```
[assembly: NeutralResourcesLanguage("en-US")]
```

When an application with that attribute runs on a system in the usual US locale, the `ResourceManager` will not attempt to search for resources. It will just go straight for the ones compiled into your main. assembly

Protection

In [Chapter 3](#), I described some of the accessibility specifiers you can apply to types and their members, such as `private` or `public`. In [Chapter 6](#), I showed some of the additional mechanisms available when you use inheritance. It's worth quickly revisiting these features, because assemblies play a part.

Also in [Chapter 3](#), I introduced the `internal` keyword and said that classes and methods with this accessibility are available only within the same *component*, a slightly vague term that I chose because I had not yet introduced assemblies. Now that it's clear what an assembly is, it's safe for me to say that a more precise description of the `internal` keyword is that it indicates that a member or type should be accessible only to code in the same assembly.⁹ Likewise, `protected internal` members are available to code in derived types, and also to code defined in the same assembly, and the similar but more restrictive `protected private` protection level makes members available only to code that is in a derived type that is defined in the same assembly.

Target Frameworks and .NET Standard

One of the decisions you need to make for each assembly that you build is the target framework or frameworks you will support. Each `.csproj` file will have either a `<TargetFramework>` element indicating the target or a `<TargetFrameworks>` element containing a list of frameworks. The particular target is indicated with a *target framework moniker* (TFM). For example, `net6.0`, `net7.0`, and `net8.0` represent .NET 6.0, .NET 7.0, and .NET 8.0, respectively. For the .NET Framework 4.6.2, 4.7.2, and 4.8, the TFMs are `net462`, `net472`, and `net48`, respectively. When you list multiple target frameworks, you will get multiple assemblies when you build, each in its own subfolder named for the TFM. The SDK effectively builds the project multiple times.

If you need access to some OS-specific functionality, there are OS-specific variants of .NET TFMs, such as `net8.0-ios` or `net8.0-windows`. (.NET Framework is inherently Windows-only.) You can incorporate an OS version number too: `net8.0-windows10.0.22621` indicates that you will be using API features introduced in Windows SDK 10.0.22621, meaning that your application can use functionality introduced in Windows 11. This doesn't mean that your component absolutely requires that version or later. It's possible to detect that you're on an older version of Windows and gracefully downgrade functionality appropriately. The OS version in the TFM determines only which APIs you can *attempt* to use.

If you need to provide different code for each target platform (perhaps because you can only implement certain functionality on newer target versions), you might need to use conditional compilation (described in [“Compilation Symbols”](#)). But in cases where the same code works for all targets, it might make sense to build for a single target, .NET Standard. As I described in [Chapter 1](#), the various versions of .NET Standard define common subsets of the .NET runtime libraries that are available across multiple versions of .NET. I said that if you need to target both .NET and .NET Framework, the best choice today is typically .NET Standard 2.0 (which has a TFM of `netstandard2.0`). However, it's worth being aware of the other options, particularly if you're looking to make your component available to the widest possible audience.

.NET libraries published on NuGet may decide to target the lowest version of .NET Standard that they can if they want to ensure the broadest reach. Versions 1.1 through 1.6 gradually added more functionality in exchange for supporting a smaller range of targets. (For example, if you want to use a .NET Standard 1.3 component on .NET Framework, it needs to be .NET Framework 4.6 or later; targeting .NET Standard 1.4 requires .NET Framework 4.6.1 or later.) .NET Standard 2.0 was a larger leap forward and marked an important point in .NET Standard's evolution: according to Microsoft's current plans, this will be the highest version number able to run on .NET Framework. Versions of .NET Framework from 4.7.2 onward fully support it, but .NET Standard 2.1 will not run on any version of .NET Framework now or in the future. It will run on all supported versions of .NET. Mono v6.4 and later support it too. But this is the end of the road for the classic .NET Framework. In practice, .NET Standard 2.0 is currently a popular choice with component authors because it enables the component to run on all recently released versions of .NET while providing access to a very broad set of features.

All of this has caused a certain amount of confusion, and you might be pleased to know that newer versions of .NET simplify things. If you don't need to support .NET Framework, you can just target .NET 6.0 or later, ignoring .NET Standard. Mono and Native AOT can run components that target .NET 6.0, so targeting .NET 6.0 will cover most runtimes. You can target later versions of .NET such as 8.0 of course; the significance of .NET 6.0 is that it was the first .NET version with long-term support in which a single target framework was available on the mobile device targets that Mono supports as well as Windows, Linux, and macOS. (Components that target .NET 6.0 run just fine on .NET 8.0 by the way. And that will continue to be true even after .NET 6.0 is out of support: as long as you're using a supported .NET runtime, it doesn't matter if your application includes components specifying a target framework that is now out of support. .NET 8.0 can happily load any component that targets any version in the .NET lineage all the way back to .NET Core 1.0.)

What does this all mean for C# developers? If you are writing code that will never be used outside of a particular project, you will normally just target the latest version of .NET. You will be able to use any NuGet package that targets .NET Standard, up to and including v2.1, and also any

package that targets any version of .NET (which means the overwhelming majority of what's on NuGet will be available to you).

If you are writing libraries that you intend to share, and if you want your components to be available to the largest audience possible, you should target .NET Standard unless you absolutely need some feature that is only available in a particular runtime. .NET Standard 2.0 is a reasonable choice—you could open your library up to a wider audience by dropping to a lower version, but today, the versions of .NET that support .NET Standard 2.0 are widely available, so you would only contemplate targeting older versions if you need to support developers still using older .NET Frameworks. (Microsoft does this in most of its NuGet libraries, but you don't necessarily have to tie yourself to the same regime of support for older versions.) Microsoft provides a useful guide to which versions of the various .NET implementations support the various [.NET Standard versions](#). If you want to use certain newer features (such as the memory-efficient types described in [Chapter 18](#)), you may need to target a more recent version of .NET Standard, with 2.1 being the latest at the time of writing, but be aware that this rules out running on .NET Framework. At that point, you might as well just target .NET 6.0 or a later version of .NET, because .NET Standard has little to offer in the unified post-.NET-Framework world. In any case, the development tools will ensure that you only use APIs available in whichever version of .NET or .NET Standard you declare support for.

Summary

An assembly is a deployable unit, almost always a single file, typically with a *.dll* or *.exe* extension. It is a container for types and code, and may also embed binary resource streams. A type belongs to exactly one assembly, and that assembly forms part of the type's identity—the .NET runtime can distinguish between two types with the same name in the same namespace if they are defined in different assemblies. Assemblies have a composite name consisting of a simple textual name, a four-part version number, a culture string, and optionally a public key token. Assemblies with a public key token are called strongly named assemblies, giving them a globally unique name. Assemblies can either be deployed along-

side the application that uses them or stored in a system-wide repository. (In .NET Framework, that repository is the Global Assembly Cache, and assemblies must be strongly named to use this. .NET provides shared copies of built-in assemblies, and depending on how you install these newer runtimes, they may also have shared copies of frameworks such as ASP.NET Core and WPF. And you can optionally set up a separate runtime package store containing other shared assemblies to avoid having to include them in application folders.)

The runtime can load assemblies automatically on demand, which typically happens the first time you run a method that contains some code that depends on a type defined in the relevant assembly. You can also load assemblies explicitly if you need to.

As I mentioned earlier, every assembly contains comprehensive metadata describing the types it contains. In the next chapter, I'll show how you can get access to this metadata at runtime.

- 1** I'm using *modern* in a very broad sense here—Windows NT introduced PE support in 1993.
- 2** With suitable build settings you can produce host executables for all supported targets regardless of which OS you build on.
- 3** This was the year Windows Vista shipped. Application manifests existed before then, but this was the first version of Windows to treat their absence as signifying legacy code.
- 4** On a conventional machine or VM this would mean running the .NET runtime installer. If you're using containers, it would mean selecting an image that includes the runtime.
- 5** This is not available in .NET Framework or .NET Standard. Isolation was typically managed with *appdomains* on .NET Framework, an older mechanism that is not supported in .NET.
- 6** If you use `Assembly.LoadFrom`, the CLR does not care whether the filename matches the simple name.

7 It's possible to configure the CLR to substitute a specific different version, but even then, the loaded assembly has to have the exact version specified by the configuration.

8 Hove, England.

9 Internal items are also available to *friend assemblies*, meaning any assemblies referred to with an `InternalsVisibleTo` attribute, as described in [Chapter 14](#).